Sequential Task:

Threads - Parking Lot Simulation

Object-Oriented Programming and GUI

05 May 2026

1. Declare a **Parking** class that represents a multi-story parking lot operated concurrently using a semaphore and atomic counters.

(a) Declare the fields:

- Add a private, immutable field **LEVELS**, which specifies the number of parking levels.
- Add a private, immutable field **SPOTS_PER_LEVEL**, which specifies the number of spaces on each level.
- Add a private, immutable field **TOTAL_CAPACITY**, which calculates the total parking capacity as the product of **LEVELS** and **SPOTS_PER_LEVEL**.
- Add a **semaphore** field of type **Semaphore**, which will control access to a limited number of parking spaces.
- Add a field, **spotsOccupiedPerLevel**, of type **AtomicInteger[]**, to store the number of occupied spaces at each level.
- Add a field, **totalOccupied**, of type **AtomicInteger**, to store the total number of currently occupied spaces.
- Add a field, **displayLock**, of type **Object**, to serve as a lock to synchronize state updates and output.
- Add a field, **totalCarsServed**, of type **AtomicInteger**, to store the total number of cars served.
- Add a field, **simulationStartTime**, of type **long**, to store the simulation start time.

(b) Implement the **Parking()** constructor, whose task is to prepare the initial state of the parking lot.

i. Create a semaphore with the number of permits equal to **TOTAL_CAPACITY**.

ii. Set the semaphore's fairness parameter to **true** so that waiting threads are handled in an orderly manner.

iii. Create an **AtomicInteger[]** array with a length equal to the number of levels (**LEVELS**).

iv. Loop through all the indices in the level array.

v. For each level, create an **AtomicInteger** counter with an initial value of 0.

vi. Create a **totalOccupied** counter with an initial value of **0**.

vii. Create a **totalCarsServed** counter with an initial value of **0**.

viii. Store the current system time in the **simulationStartTime** field using **System.currentTimeMillis()**.

(c) Implement the **enter(String carNumber)** method that handles a car entering the parking lot and returns the level where the car was parked.

i. Take a **carNumber** parameter that identifies the car attempting to enter the parking lot.

ii. Display a message indicating that the car is waiting for a free space.

iii. Call **semaphore.acquire()** to block the thread until a free space is available.

iv. Determine the parking level with available spaces by calling the **findFreeLevel()** method and storing the result in the level variable.

v. Enter the **synchronized(displayLock)** block to atomically update the parking status and display the information.

A. Increment the occupied space counter for the selected level by calling **spotsOccupiedPerLevel[level].incrementAndGet()**.

B. Increment the total occupied space counter by calling **totalOccupied.incrementAndGet()**.

C. Increment the total number of cars served by calling **totalCarsServed.incrementAndGet()**.

D. Display a message indicating that a car has parked at **level + 1**.

E. Call the **displayStatus()** method to show the current parking status.

vi. Return the level index so that it can be used later when cars leave.

(d) Implement the exit(String carNumber, int level) method to handle the car's exit from the parking lot.

i. Accept a carNumber parameter that identifies the car leaving the parking lot.

ii. Accept a level parameter that specifies the level occupied by the car.

iii. Enter the **synchronized(displayLock)** block to synchronize the modification of the parking lot status and the display of messages.

A. Decrement the number of occupied spaces at the specified level by calling **spotsOccupiedPerLevel[level].decrementAndGet()**.

B. Decrement the total number of occupied spaces by calling **totalOccupied.decrementAndGet()**.

C. Release one semaphore permission by calling **semaphore.release()**.

D. Display a message informing that the car has left the specified parking lot level.

E. Call the **displayStatus()** method to show the current parking lot status.

(e) Implement a private method **findFreeLevel()**, whose task is to find the first level with a free space.

i. Traverse all parking levels from index **0** to **LEVELS – 1**.

ii. For each level, retrieve the number of occupied spaces by calling **spotsOccupiedPerLevel[i].get()**.

iii. Check if the number of occupied spaces is less than **SPOTS_PER_LEVEL**.

iv. If the level has a free space, return its index.

v. If no level is specified in the loop, return **0** as a fallback value.

(f) Implement a private method **displayStatus()**, whose task is to display the current occupancy status of the parking lot.

i. Display the parking status section header.

ii. Traverse all parking levels.

A. Get the number of occupied spaces on the current level by calling **spotsOccupiedPerLevel[i].get()**.

B. Display the beginning of the line describing the parking level and the opening character of the space visualization.

C. Loop through all spaces on a given level from **0** to **SPOTS_PER_LEVEL-1**.

D. For spaces with an index less than the number of occupied spaces, print the character "X".

E. For the remaining spaces, print the character "_".

F. Terminate the line with numerical information in the format "occupied spaces for all spaces on that level".

iii. Display the total number of occupied spaces as the value of **totalOccupied.get()** relative to **TOTAL_CAPACITY**.

iv. Display the bottom separator for the status section and a blank line.

(g) Implement the **displayStatistics()** method, which displays the final statistics of the parking simulation.

i. Calculate the simulation duration as the difference between the current system time and the **simulationStartTime** value.

ii. Store the result in the **simulationTime** variable.

iii. Display the header of the final statistics section.

iv. Display the total simulation time in seconds by dividing **simulationTime** by **1000.0**.

v. Display the total number of cars served by reading **totalCarsServed.get()**.

vi. Display the number of available spaces by calling **semaphore.availablePermits()** and comparing it with **TOTAL_CAPACITY**.

vii. Display the bottom separator of the statistics section.

2. Declare a **Car** class that implements the **Runnable** interface and represents a car running as a separate thread in the parking simulation.

(a) Declare the fields:

• Add a private field **number** of type **String** that stores the car's ID number.

• Add a private field **parking** of type **Parking** that stores a reference to the parking lot handling car entry and exit.

• Add a private field **random** of type **Random** that will be used to generate random delays in the simulation.

– Initialize the field with a new **Random** instance.

(b) Implement the **Car(String number, Parking lot)** constructor, which initializes the car data.

i. Accept a number parameter representing the car number.

ii. Accept a parking parameter representing the parking lot used by the car.

iii. Assign the passed values to the number and parking fields.

(c) Implement the **run()** method, which defines the complete lifecycle of the car in the simulation.

i. Place all the execution logic in a **try** block to handle possible thread interruptions.

ii. Pause the thread for a random amount of time before attempting entry.

• Call **Thread.sleep(random.nextInt(3000))**.

- Obtain a delay between **0** and less than **3000** milliseconds.

iii. Call **parking.enter(number)** to enter the car into the parking lot.

iv. Store the returned parking level index in the **level** variable.

v. Calculate a random parking time and store it in the variable **parkingTime**.

- Use the expression **2000 + random.nextInt(3000)**.

- Obtain a parking time from **2000** to less than **5000** milliseconds.

vi. Pause thread execution for the time stored in **parkingTime**.

vii. Call the **parking.exit(number, level)** method to exit the parking lot from the saved level.

viii. Handle the **InterruptedException** exception.

- Restore the interrupted status of the current thread by calling **Thread.currentThread().interrupt()**.

- Display an error message containing the car number and information about the interruption.

3. Verify the program outputs:

```
>>> PARKING SIMULATION START <<<
Car-02 is waiting for a spot...
[+] Car-02 parked on level 1
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 1/15
======================
Car-13 is waiting for a spot...
[+] Car-13 parked on level 1
=== PARKING STATUS ===
Level 1: [XX___] (2/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 2/15
======================
Car-11 is waiting for a spot...
[+] Car-11 parked on level 1
=== PARKING STATUS ===
Level 1: [XXX__] (3/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 3/15
======================
```

Car-04 is waiting for a spot...
[+] Car-04 parked on level 1
=== PARKING STATUS ===
Level 1: [XXXX_] (4/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 4/15
==========
Car-03 is waiting for a spot...
[+] Car-03 parked on level 1
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 5/15
==========
Car-06 is waiting for a spot...
[+] Car-06 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [X____] (1/5)
Level 3: [_____] (0/5)
Occupied spots: 6/15
==========
Car-15 is waiting for a spot...
[+] Car-15 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XX___] (2/5)
Level 3: [_____] (0/5)
Occupied spots: 7/15
==========
Car-18 is waiting for a spot...
[+] Car-18 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXX__] (3/5)
Level 3: [_____] (0/5)
Occupied spots: 8/15
==========
Car-19 is waiting for a spot...
[+] Car-19 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXX_] (4/5)
Level 3: [_____] (0/5)
Occupied spots: 9/15
==========
Car-16 is waiting for a spot...
[+] Car-16 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)

Level 2: [XXXXX] (5/5)
Level 3: [_____] (0/5)
Occupied spots: 10/15
==========
Car-07 is waiting for a spot...
[+] Car-07 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [X____] (1/5)
Occupied spots: 11/15
==========
Car-09 is waiting for a spot...
[+] Car-09 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XX___] (2/5)
Occupied spots: 12/15
==========
Car-01 is waiting for a spot...
[+] Car-01 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXX__] (3/5)
Occupied spots: 13/15
==========
Car-17 is waiting for a spot...
[+] Car-17 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXX_] (4/5)
Occupied spots: 14/15
==========
Car-08 is waiting for a spot...
[+] Car-08 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15
==========
Car-20 is waiting for a spot...
Car-12 is waiting for a spot...
Car-05 is waiting for a spot...
Car-10 is waiting for a spot...
Car-14 is waiting for a spot...
[-] Car-06 left from level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)

Level 2: [XXXX_] (4/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 14/15
==========
[+] Car-20 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15
==========
[-] Car-18 left from level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXX_] (4/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 14/15
==========
[+] Car-12 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15
==========
[-] Car-19 left from level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXX_] (4/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 14/15
==========
[+] Car-05 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15
==========
[-] Car-15 left from level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXX_] (4/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 14/15
==========
[+] Car-10 parked on level 2
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15

==========
[-] Car-07 left from level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXX_] (4/5)
Occupied spots: 14/15
==========
[+] Car-14 parked on level 3
=== PARKING STATUS ===
Level 1: [XXXXX] (5/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 15/15
==========
[-] Car-02 left from level 1
=== PARKING STATUS ===
Level 1: [XXXX_] (4/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 14/15
==========
[-] Car-13 left from level 1
=== PARKING STATUS ===
Level 1: [XXX__] (3/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 13/15
==========
[-] Car-04 left from level 1
=== PARKING STATUS ===
Level 1: [XX___] (2/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 12/15
==========
[-] Car-11 left from level 1
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXXX] (5/5)
Occupied spots: 11/15
==========
[-] Car-17 left from level 3
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXXX_] (4/5)
Occupied spots: 10/15
==========
[-] Car-08 left from level 3
=== PARKING STATUS ===

Level 1: [X____] (1/5)
Level 2: [XXXXX] (5/5)
Level 3: [XXX__] (3/5)
Occupied spots: 9/15
==========
[-] Car-05 left from level 2
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [XXXX_] (4/5)
Level 3: [XXX__] (3/5)
Occupied spots: 8/15
==========
[-] Car-09 left from level 3
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [XXXX_] (4/5)
Level 3: [XX___] (2/5)
Occupied spots: 7/15
==========
[-] Car-01 left from level 3
=== PARKING STATUS ===
Level 1: [X____] (1/5)
Level 2: [XXXX_] (4/5)
Level 3: [X____] (1/5)
Occupied spots: 6/15
==========
[-] Car-03 left from level 1
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [XXXX_] (4/5)
Level 3: [X____] (1/5)
Occupied spots: 5/15
==========
[-] Car-16 left from level 2
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [XXX__] (3/5)
Level 3: [X____] (1/5)
Occupied spots: 4/15
==========
[-] Car-14 left from level 3
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [XXX__] (3/5)
Level 3: [_____] (0/5)
Occupied spots: 3/15
==========
[-] Car-12 left from level 2
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [XX___] (2/5)
Level 3: [_____] (0/5)

Occupied spots: 2/15
==========
[-] Car-20 left from level 2
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [X____] (1/5)
Level 3: [_____] (0/5)
Occupied spots: 1/15
==========
[-] Car-10 left from level 2
=== PARKING STATUS ===
Level 1: [_____] (0/5)
Level 2: [_____] (0/5)
Level 3: [_____] (0/5)
Occupied spots: 0/15
==========
>>> SIMULATION END <<<
=== FINAL STATISTICS ===
Total simulation time: 8.378 seconds
Total cars served: 20
Available spots: 15/15
==========

4. Milestone 2: present the result of the work to the instructor.